Table of Contents

1. Fundamentals

- 1.1. Object Oriented Programming
- 1.2. Aspect Oriented Programming
- 1.3. Functional Programming
- 1.4. Reactive Programming
- 1.5. SOLID Principles
- 1.6. Design Patterns
- 1.7. Data Structures
- 1.8. Algorithms

2. Database

- 2.1. Relational Databases
- 2.2. Non-relational Databases
- 2.3. Advanced Architectural Concepts
- 2.4. Query Optimization

3. Operating Systems

- 3.1. Process Management
- 3.2. Inter-Process Communication
- 3.3. Memory Management
- 3.4. Storage and File Systems

4. System Design

- 4.1. Architectural Approach
- 4.2. Interview Techniques

5. Modern Architecture

- 5.1. Microservices
- 5.2. Distributed Systems
- 5.3. Container & Orchestration
- 5.4. CI/CD
- 5.5. Cloud

- 5.6. Version Control

6. Java

- 6.1. Java 7
- 6.2. Java 8
- 6.3. Java 11
- 6.4. Java 17
- 6.5. Java 21
- 6.6. Spring Boot
- 6.7. Memory Management

7. Mobile

- 7.1. Android Native
- 7.2. iOS Native
- 7.3. Flutter
- 7.4. React Native

8. UI

- 8.1. HTML
- 8.2. CSS
- 8.3. JavaScript
- 8.4. React.js

9. Testing

- 9.1. Unit Testing
- 9.2. Integration Tests
- 9.3. Test Driven Development

10. Network

- 10.1. OSI Model
- 10.2. HTTP/HTTPS
- 10.3. DNS & Load Balancing
- 10.4. Protocols

11. Security

- 11.1. OWASP Top 10

- 11.2. Authentication vs. Authorization
- 11.3. OAuth2 & OpenID Connect
- 11.4. Encryption

12. Software Development Life Cycle

- 12.1. Agile/Scrum
- 12.2. Code Review Culture
- 12.3. Git Flow
- 12.4. Documentation

13. AI

- 13.1. Machine Learning Fundamentals
- 13.2. Deep Learning
- 13.3. NLP & GenAI
- 13.4. Implementation & Ethics

Fundamentals

Object_Oriented_Programming

Object-Oriented Programming (OOP) Principles

OOP is a programming paradigm based on the concept of **"objects"**, which can contain data (attributes) and code (methods). It aims to implement real-world entities like inheritance, hiding, and polymorphism in programming. It uses classes as blueprints to create reusable, modular, and organized code, representing real-world entities. Key, foundational pillars include encapsulation, inheritance, polymorphism, and abstraction.

Real-World Analogy: In the real world, every object has properties, functions, and a state. Similarly, in OOP:
- **Attributes (Properties):** Define what the object *is* (e.g., Brand, Color).
- **Functions (Methods):** Define what the object *does* (e.g., Drive, Brake).
- **State:** Defines the object's condition at a specific time (e.g., Current Speed, Fuel Level).

---

1. Encapsulation (Data Hiding)

Encapsulation is the bundling of data and the methods that operate on that data into a single unit (class). It restricts direct access to some of an object's components.

- **Key Concept:** Use `private` variables and `public` getters/setters.
- **Why?** It protects the internal state of an object from unauthorized access and keeps the code modular.
- **Interview Tip:** If asked "Why not just use public variables?", answer: *"To maintain control over data validation and protect the object's integrity (Invariants)."*

---

2. Inheritance

Inheritance allows a class (subclass/child) to acquire the properties and behaviors of another class (superclass/parent).

- **Key Concept:** The `extends` keyword in Java.
- **Why?** It promotes **code reusability** and establishes a "is-a" relationship.

---

3. Polymorphism (Many Forms)

Polymorphism allows objects of different classes to be treated as objects of a common superclass.

Types:
1. **Static (Compile-time):** Method Overloading (same name, different parameters). Multiple methods in the same class share the same name but have different parameters (type or number)
2. **Dynamic (Runtime):** Method Overriding. A child class provides a specific implementation of a method already defined in its parent class. The method to be executed is determined at runtime based on the object type, not the reference type
3. **Subtype Polymorphism**: A function can work with a base class interface, while the runtime ensures the correct overridden method runs for the specific subclass
Example: A single `Shape` interface with `draw()` implemented differently by `Circle`, `Square`, and `Triangle` classes.
Benefits:
- **Flexibility:** Easily add new subclasses without changing existing code.
- **Code Maintenance:** Reduces complexity by allowing generic code to handle various types.
- **Consistency:** A uniform interface is used for different underlying forms.

4. Abstraction

Abstraction is the process of hiding the implementation details and showing only the essential features of the object. It reduces complexity, improves maintainability, and enables users to interact with objects through simplified, high-level interfaces

- **Key Concept:** `abstract` classes and `interfaces`.
- **Simplifying Complexity:** By focusing on what an object does rather than how it does it, code becomes more manageable.
- **Real-World Analogy:** A car driver uses a steering wheel and pedals (the interface) without needing to understand the intricate details of the internal engine mechanism.
- **Interview Tip:** "When to use an Interface vs. an Abstract Class?"
  - Use **Interface** for "can-do" (behavior) capabilities.
  - Use **Abstract Class** for "is-a" (identity) relationships.
- **Abstract Classes:** These act as blueprints, restricting the instantiation of objects and enforcing structure on subclasses.
- **Interfaces:** Similar to abstract classes, these define a contract that implementing classes must follow.
- **Benefits:** Promotes modularity, enhances code readability, and facilitates easier maintenance.

Interview Questions

1. **What is the difference between Abstraction and Encapsulation.**
   - Abstraction is a design-level process that hides *complexity*
   - Encapsulation is an implementation-level mechanism that hides *data* (using private fields)
2. **What is the "Diamond Problem" in OOP, and how does Java handle it?**
   The **Diamond Problem** is an ambiguity that arises when a class inherits from two or more classes that have a common ancestor. Imagine a base class A with a method performAction(). Classes B and C both inherit from A and override performAction() with their own specific logic. If Class D were allowed to inherit from both B and C (Multiple Inheritance), and we called d.performAction(), the compiler wouldn't know which version to execute—the one from B or the one from C ?
   Java solves the Diamond Problem through a two-layered approach that prioritizes simplicity and predictability:

1. **Class Level: Strict Single Inheritance**
   Java strictly prohibits a class from extending more than one class. This design choice fundamentally eliminates the possibility of the Diamond Problem in the traditional sense, as there is only ever one path for state and behavior inheritance between classes.
2. **Interface Level: Explicit Conflict Resolution (Java 8+)**
   With the introduction of Default Methods in Java 8, a class can inherit conflicting method implementations from two different interfaces. Java resolves this using the following three rules:

- **Rule 1: Class Wins Over Interface**
  If a class extends a base class and implements an interface that both provide a method with the same signature, the implementation in the class (or its parent) always takes precedence.
- **Rule 2: Sub-interface Wins Over Super-interface**
  If interface B extends interface A, and both have a default method, the implementation in the more specific interface (B) is chosen.
- **Rule 3: Explicit Override (The Manual Resolution)**
  If two independent interfaces (InterfaceA and InterfaceB) provide the same default method and there is no inheritance relationship between them, the compiler will throw a Compile-Time Error. The developer must resolve the ambiguity manually by overriding the method in the subclass.

3. **What is the difference between an Abstract Class and an Interface in Modern Java?**
   Abstract classes could have state and behavior, while Interfaces could only have method signatures. Since **Java 8, 9, and 11**, this boundary has blurred, but key architectural differences remain.

   - A class can extend only one abstract class. A class can implement multiple interfaces.
   - Abstract class can have instance variables (fields) to maintain state. Interface cannot have instance variables (only public static final constants).
   - Abstract class methods can have any access modifier (private, protected, etc.). Interface methods are public by default (private methods allowed since Java 9).
   - Abstrace class can have constructors. Interfaces cannot have constructors.
   - Abstract class provides a base for "is-a" relationships. Interface provides a contract for "can-do" capabilitie
4. **What is the "Fragile Base Class" problem?**
   The **Fragile Base Class** problem is an architectural pitfall in OOP where modifications to a base class (superclass) unintentionally break the functionality of derived classes (subclasses).

5. **Why "Fragile Base Class" problem happens?**
   - **Tight Coupling:** Subclasses depend on the internal implementation details of the base class.
   - **Inappropriate Overriding:** A developer changes a method in the base class to call another method, which the subclass has already overridden, leading to infinite loops or unexpected side effects.
6. **What is the solution for "Fragile Base Class" problem?**
   Modern software engineering suggests favoring **Composition over Inheritance**. By using composition, you limit the exposure of internal logic and reduce the risk of breaking downstream components
7. **What is the difference between Composition and Inheritance?**
   These are two ways to reuse code and define relationships between classes.
   - **Inheritance (Is-A):**
     - Represents a strict hierarchy (e.g., `Car` **is a** `Vehicle`).
     - Achieved via the `extends` keyword.
     - **Cons:** Creates tight coupling; changes in the parent affect all children.
   - **Composition (Has-A):**
     - Represents a relationship where one class contains an instance of another (e.g., `Car` **has an** `Engine`).
     - Achieved by defining a member variable of another class type.
     - **Pros:** Flexible and decoupled. You can swap the "Engine" at runtime (Dependency Injection).
       Senior Principle: Always start with **Composition**. Only move to **Inheritance** if you are certain that a strict, permanent hierarchical relationship exists and that you need to take advantage of polymorphism across that specific hierarchy.

Aspect_Oriented_Programming

Aspect-Oriented Programming (AOP)

AOP is a programming paradigm that aims to increase modularity by allowing the separation of **cross-cutting concerns**. While OOP organizes code into vertical hierarchies (Classes/Objects), AOP allows us to handle horizontal concerns that affect multiple, unrelated classes.

---

1. The Core Problem: Tangling & Scattering

Before AOP, cross-cutting concerns (like logging or security) had to be manually added to every function. This led to two major issues:

- **Code Tangling:** Business logic is mixed with infrastructure logic (e.g., a "Transfer Money" method is 2 lines of logic and 20 lines of logging/validation).
- **Code Scattering:** The same infrastructure code is duplicated across dozens of different classes, making it a nightmare to update.

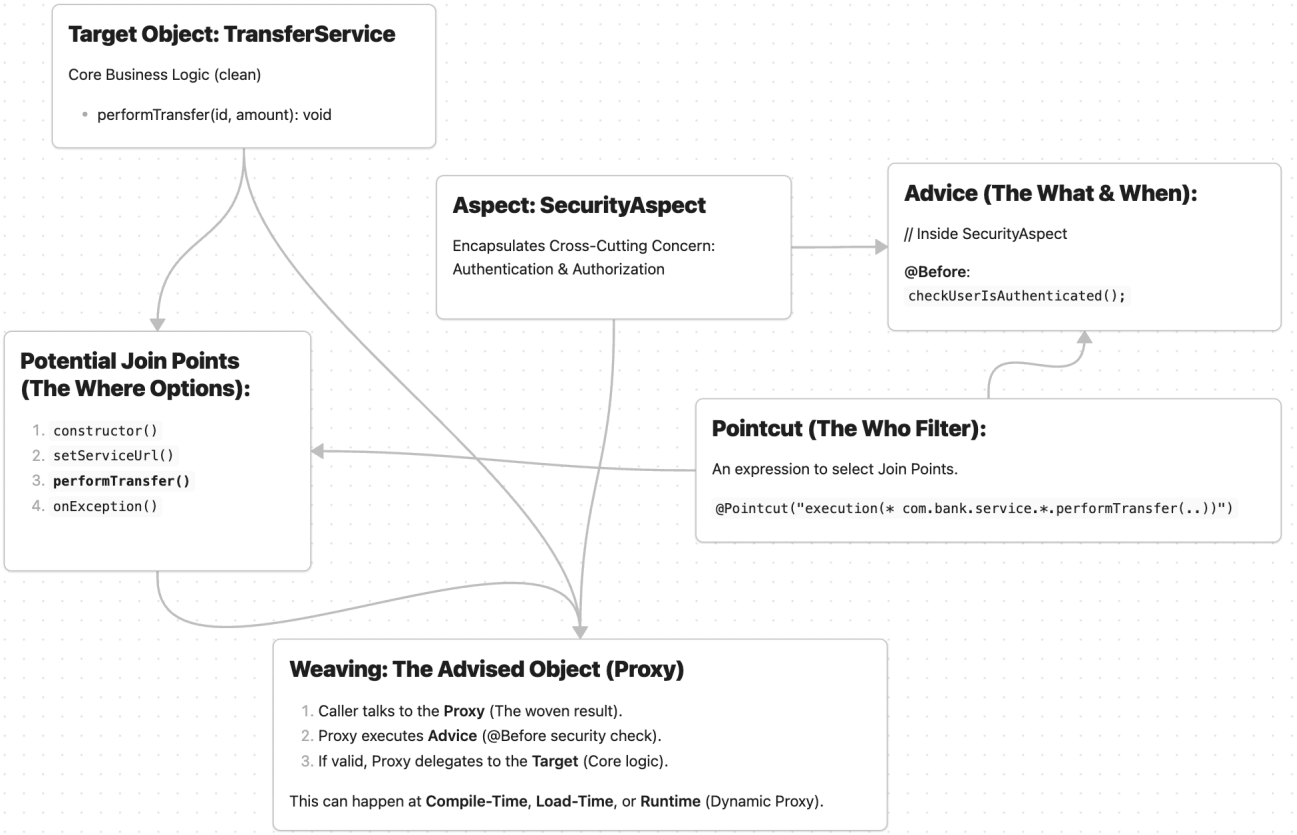
---

2. AOP Concepts (The Vocabulary of the Paradigm)

Regardless of the language or framework, AOP uses a specific set of concepts to "weave" logic into your code:

- **Aspect:** The module that encapsulates a cross-cutting concern (e.g., `LoggingAspect`).
- **Join Point:** A specific point during program execution where an aspect could be plugged in (e.g., a method call, a constructor, or even an exception being thrown).
- **Advice:** The code that is executed at a Join Point. It defines **what** happens and **when** (Before, After, or Around the event).
- **Pointcut:** A filter/expression that defines exactly which Join Points to target.
- **Target Object:** The object being advised by one or more aspects.
- **Weaving:** The technical process of linking the Aspect with the Target Object. This can happen at:
  - **Compile-Time:** The code is modified during the build process.
  - **Load-Time:** The code is modified when the class is loaded into memory.
  - **Runtime:** A proxy is created at execution time (Common in Spring).

AOP Example:



Terminology is confusing here so explanation with an analogy.
Join Point Every door/window in the building. Any point in the program execution (method call, etc.). Where can I plug in?
Pointcut The rule: "Only 1st-floor doors." An expression that filters/selects Join Points. Which ones do I care about?
Think like list of all joining points that you can interrupt and a fraction of them like cutting a pa

---

3. AOP Implementation Strategies

AOP is typically implemented using one of two major strategies:

1. **Bytecode Manipulation (Static Weaving):**
   - The AOP engine modifies the actual compiled bytecode of your classes.
   - **Pros:** High performance; can intercept private methods and constructors.
   - **Example:** AspectJ.
2. **Proxy-based (Dynamic Weaving):**
   - The system creates a "wrapper" (proxy) around the target object. The caller talks to the proxy, which executes the aspect logic before/after delegating to the real object.

- **Pros:** Easier to implement; doesn't require a custom compiler.
- **Example:** Spring AOP, Python Decorators.

Interview Questions

1. **Why is AOP considered a "complement" to OOP rather than a replacement?**
   OOP is excellent for modeling core business entities and their relationships (Vertical structure). However, OOP fails at managing logic that is common to many unrelated entities (Horizontal structure). AOP fills this gap by allowing us to modularize infrastructure logic that would otherwise "pollute" our clean OOP hierarchies.
2. **What are the risks of using AOP in a large-scale system?**
   - **Hidden Control Flow:** Since the logic is "woven" in, it isn't visible in the source code of the business method. This can make debugging and tracing very difficult.
   - **Performance Overhead:** Especially with runtime proxies, there is a small cost for every intercepted call.
   - **Fragility:** Pointcuts often rely on naming conventions (e.g., `execution(* com.service.*.*(..))`). If a developer renames a package or method, the aspect might silently stop working.
3. **How does AOP handle exceptions thrown by the business logic?**
   AOP provides "After Throwing" advice. If the business logic throws an exception, the aspect catches it, allowing for centralized logging, transaction rollbacks, or even "exception translation" (converting a low-level SQL error into a high-level Business error) before re-throwing it.

Tip

Senior Insight: In a professional environment, AOP should be used to make business logic **pure**. If you find yourself writing an Aspect that changes the actual state of an object or influences business decisions, you are probably misusing the paradigm.
Senior Insight: AOP is powerful but dangerous. Overusing it makes the code "invisible" and hard to debug. Use AOP for **infrastructure concerns** (logging, metrics, transactions), but avoid putting core **business logic** into aspects.

Functional Programming (FP)

Functional Programming is a declarative programming paradigm where programs are constructed by applying and composing **functions**. It treats computation as the evaluation of mathematical functions and avoids **changing-state** and **mutable data**.

---

1. Core Principles of FP

To master FP, you must understand these five "pillars" that differ from the traditional OOP approach:

First-Class and Higher-Order Functions

Functions are "first-class citizens," meaning they can be assigned to variables, passed as arguments, and returned from other functions.
Higher-Order Function: A function that takes another function as a parameter (e.g., `.map()`, `.filter()`).

Pure Functions

A function is "pure" if:

1. It always returns the same output for the same input.
2. It has **no side effects** (it doesn't change global variables, write to disk, or modify the input).

Immutability

In FP, data does not change. Instead of modifying an existing object, you create a **new** object with the updated values. This eliminates a whole category of bugs related to "shared mutable state."

Declarative vs. Imperative

Imperative: You tell the computer *how* to do it (loops, state changes).
Declarative: You tell the computer *what* you want (transformations).

---

2. Functional Programming in Modern Java (Streams API)

Java 8+ brought FP to the forefront of the ecosystem. Here is how we translate OOP logic into FP logic:

The OOP Way (Imperative):

```
List<String> result = new ArrayList<>();
for (User u : users) {
    if (u.getAge() > 18) {
        result.add(u.getName().toUpperCase());
    }
}
```

The FP Way (Declarative):

```
List<String> result = users.stream()
     .filter(u -> u.getAge() > 18) // Predicate
     .map(u -> u.getName().toUpperCase()) // Function
     .collect(Collectors.toList());
```

3. Key FP Vocabulary

- **Lambda Expression:** An anonymous function (e.g., `(x, y) -> x + y`).
- **Closure:** A function that "remembers" the environment in which it was created.
- **Currying:** Breaking down a function that takes multiple arguments into a series of functions that each take a single argument.
- **Memoization:** An optimization technique where you store the results of expensive function calls and return the cached result when the same inputs occur again.

Interview Questions

1. **What is a "Side Effect" in Functional Programming?**
   A side effect is any change in the state of the application that is observable outside the

called function other than its return value. Examples include:

- Modifying a global variable.
- Modifying an argument passed by reference.
- Printing to the console or saving to a database.
- Throwing an exception. In "Pure" FP, we try to isolate side effects to the very edges of the application.

2. **Why is Immutability important in Multi-threaded environments?**
   Immutability is the ultimate solution to **Thread Safety**. If an object cannot change after it is created, multiple threads can read it simultaneously without the risk of a "Race Condition." You don't need `synchronized` blocks or locks because there is no "shared mutable state" to protect.
3. **What is the difference between *map* and *flatMap*?**
   map: Transforms each element in a stream into exactly one other element. **flatMap:** Used when each element in the original stream can be transformed into **another stream** (or list). It then "flattens" those multiple streams into a single stream. Analogy: *map* is like painting every house on a street; *flatMap* is like opening the front door of every house and bringing everyone inside out into the street.

Tip

Senior Insight: Don't try to make everything "Pure." In real-world systems, you need side effects to save data and show UI. The goal is to keep your **Business Logic** functional and pure, while pushing the **I/O and State** to the infrastructure layer.

Reactive_Programming

Reactive Programming

Reactive programming is a declarative programming paradigm concerned with **data streams** and the **propagation of change**. In this model, you don't call a service and wait for a response; you subscribe to a stream and "react" whenever new data is emitted.

---

1. The Core Pillars: The Reactive Manifesto

To be truly "Reactive," a system should aim to satisfy these four traits:

1. **Responsive:** The system responds in a timely manner.
2. **Resilient:** The system stays responsive in the face of failure (via replication/isolation).
3. **Elastic:** The system stays responsive under varying workload (scaling).
4. **Message Driven:** Uses asynchronous message-passing to ensure loose coupling.

---

2. Key Concepts

Streams (Observables/Flowables)

Everything is a stream. A stream is a sequence of ongoing events ordered in time. It can emit three things:

- **Value:** The actual data.
- **Error:** If something goes wrong.
- **Completed Signal:** When the stream ends.

Backpressure

This is the most "Senior" concept in Reactive Programming.
Problem: What happens if a "Fast Producer" sends 10,000 items per second, but a "Slow Consumer" can only process 100?
Solution: Backpressure is the mechanism where the consumer signals to the producer to "slow down" so the system doesn't run out of memory.

Schedulers & Threading

Reactive code is often asynchronous. Schedulers allow you to specify which thread pool a specific task should run on (e.g., `Schedulers.io()` for DB calls vs. `Schedulers.main()` for UI updates in Flutter/Android).

---

3. Reactive Programming in the Real World

In Flutter/Dart:

Dart uses **Streams** natively. When you use the `StreamBuilder` widget or the **BLoC (Business Logic Component)** pattern, you are practicing Reactive Programming. You react to state changes rather than manually updating the UI.

In Java/Spring:

Project Reactor and **Spring WebFlux** allow for non-blocking I/O. Instead of "one thread per request," a small number of threads can handle thousands of concurrent connections by never sitting idle while waiting for the database.

The Evolution of Reactive Programming

The history of Reactive Programming is a journey from **Functional Math** to **Asynchronous Data Streams**.

1. The Academic Roots (1970s - 1990s)

The concept started with **Dataflow Programming** and Functional Reactive Programming (FRP).

- **The Spreadsheet Analogy:** The earliest "Reactive" system most people used was **Microsoft Excel**. If cell `A1` changes, cell `B1` (which contains `=A1*2`) updates **automatically**. You don't "call" B1 to update; it *reacts* to the change in A1.

2. The Observer Pattern & Reactive Extensions (2000s - 2010)

- **The Observer Pattern (GoF):** This was the OOP way to do reactivity, but it was clunky and led to "Callback Hell."
- **Rx (Reactive Extensions):** Erik Meijer at Microsoft created **Rx.NET** in 2009. This was the "Big Bang" for modern AOP. It treated events as collections (Streams) that you could `map`, `filter`, and `reduce`. This soon spread to **RxJava**, **RxJS**, and **RxDart**.

3. The "Real-time" Revolution: Meteor.js (2012)

Meteor.js was a pioneer because it implemented **Transparent Reactivity**.

- It used a "Distributed Data Protocol" (DDP).

- If you updated a document in MongoDB, Meteor's "Tracker" would automatically re-run the functions on the client side.
- **Why it faded:** It was a "Black Box" that was hard to scale and very resource-heavy because the server had to track every single user's state.

4. The Component Era: React, Vue, and Angular (2013 - Present)

Modern frontend frameworks took the "Reactive" concept but applied it specifically to the **DOM (Document Object Model)**.

- **React.js (2013):** Introduced the **Virtual DOM**. It isn't "fully reactive" in the Rx sense; instead, it uses a "Pull" model where it re-renders components when `state` or `props` change. It made the UI a pure function of the state: `UI = f(state)`.
- **Vue.js (2014):** Evan You took a more "Meteor-like" approach but made it lightweight. Vue uses **Getters/Setters (Proxies)** to "track" dependencies. When you touch a variable, Vue "records" that the component depends on it. When the variable changes, it triggers a re-render.
- **Angular (2016+):** Heavily integrated **RxJS**. In Angular, many core features (like HTTP requests and Forms) are literal Observables.

5. The Mobile/Backend Era: Flutter & WebFlux (2018 - Present)

- **Flutter:** Uses **Streams** and **Sinks** as a first-class citizen. The **BLoC** pattern is essentially "Pure Reactive Programming" for mobile apps.
- **Spring WebFlux:** Brought the Reactive paradigm to the Java backend to handle high-concurrency without needing thousands of threads.

Interview Questions

1. **What is the difference between "Cold" and "Hot" Observables?**
   Cold Observables: Start emitting data only when someone subscribes. Each subscriber gets its own independent copy of the data from the beginning (e.g., a File download).
   Hot Observables: Emit data even if no one is listening. Subscribers only see the data emitted after they joined (e.g., a Live Stream or a Mouse Click event).
2. **Why use Reactive Programming instead of standard Multithreading?**
   Standard multithreading (Imperative) is expensive. Each thread consumes significant memory (stack size). When threads wait for I/O (Database/API), they are wasted

resources.
Reactive Programming uses an "Event Loop" (like Node.js or Vert.x). Threads never wait; they are immediately released to do other work until the I/O signal returns, allowing for much higher **scalability** with fewer resources.

3. **Explain the concept of "Operators" in Reactive Programming.**
   Operators are the "tools" used to transform, filter, and combine streams.
   - **map:** Transforms items.
   - **filter:** Discards items that don't match a criteria.
   - **debounce:** Only emits an item if a specific timespan has passed without another emission (critical for "Search-as-you-type" features).
   - **zip:** Combines the results of two different streams into one.
4. **Is React.js actually "Reactive Programming"?**
   Technically, **no**. In true Reactive Programming (like RxJS), changes are "pushed" through a pipeline. React uses a "Schedule and Re-render" approach. You change the state, and React decides when and how to "diff" the Virtual DOM to update the UI. However, the *mental model* is reactive: you change data, and the UI eventually "reacts" to it.
5. **What did Meteor.js teach the industry about Reactivity?**
   Meteor proved that **Full-stack Reactivity** is incredibly powerful for Developer Experience (DX) but dangerous for performance. It led to the rise of more granular reactive systems (like **Svelte** or **SolidJS**) that update only the specific part of the DOM that changed, rather than re-running entire component trees.
6. **How does "Signal-based" reactivity (Vue/Svelte/Angular 16) differ from "Stream-based" (RxJS)?**
   Stream-based (RxJS): Focuses on the *event* and the *transformation* over time (e.g., "A stream of clicks").
   Signal-based: Focuses on the *value*. A Signal is a piece of data that "knows" who is using it. When the value changes, it directly notifies only the consumers. Signals are generally easier to read and more performant for UI updates than complex RxJS observables.

Tip

Senior Insight: Reactive programming introduces a "Steep Learning Curve." It makes stack traces much harder to read because the execution is asynchronous. Only use it when you truly need high concurrency or are dealing with continuous data streams; don't use it for simple CRUD apps where standard Imperative code is more readable.

> **Tip**
>
> **Senior Insight:** The industry is currently moving from "Stream-heavy" (RxJS) back toward "Signal-based" reactivity (Vue, Preact Signals, Angular Signals). Why? Because streams are great for complex asynchronous logic, but **Signals** are much more intuitive for managing simple UI state without the "Marble Diagram" headache.

```
Dataview (for inline query '=A1*2'): No implementation found for
'null * number'
```

SOLID_Principles

SOLID Principles

SOLID is a mnemonic acronym for five design principles intended to make software designs more understandable, flexible, and maintainable. These principles are the backbone of professional software engineering.

---

1. S: Single Responsibility Principle (SRP)

> "A class should have one, and only one, reason to change."

The Core: A class should do one thing and do it well. If a class has multiple responsibilities, they become coupled, leading to fragile designs.

- **Bad:** A `User` class that handles data validation, database persistence, and email notifications.
- **Good:** Split into `User` (Model), `UserRepository` (Persistence), and `EmailService` (Communication).

---

2. O: Open/Closed Principle (OCP)

> "Software entities should be open for extension, but closed for modification."

The Core: You should be able to add new functionality without touching existing, tested code.

- **Method:** Use Interfaces and Abstraction.
- **Example:** Instead of adding an `if-else` inside a `PaymentProcessor` for every new currency, create a `PaymentStrategy` interface. New currencies just implement the interface without changing the processor.

---

3. L: Liskov Substitution Principle (LSP)

> "Objects of a superclass should be replaceable with objects of its subclasses without breaking the application."

The Core: A subclass must fulfill the "contract" of its parent. It should not withdraw or change expected behaviors.

- **Classic Violation:** The Square-Rectangle problem. If a `Square` inherits from `Rectangle` but breaks the logic of independent width/height setting, it violates LSP.
- **Interview Tip:** "Is-a" relationship is not enough; "Behavioral Compatibility" is mandatory.

---

4. I: Interface Segregation Principle (ISP)

> "Clients should not be forced to depend upon interfaces that they do not use."

The Core: Better to have many specific interfaces than one general-purpose "Fat Interface."

- **Example:** If you have an `IMachine` interface with `print()`, `scan()`, and `fax()`, a simple printer shouldn't be forced to implement `fax()`. Split them into `IPrinter`, `IScanner`, and `IFax`.

---

5. D: Dependency Inversion Principle (DIP)

> "High-level modules should not depend on low-level modules. Both should depend on abstractions."

The Core: Depend on **Interfaces**, not **Concrete Classes**. This decouples your system components.

- **Example:** A `Notification` class should not depend on `GmailService` directly. It should depend on an `IMessageService` interface. This allows you to swap Gmail for SendGrid or SMS without changing the `Notification` logic.

Interview Questions

1. How is Lyskov Substitution Principle different from simple Polymorphism?
   Polymorphism is a language feature (how to override); LSP is a design rule (when to override). LSP ensures that the derived class is a true "subtype" that doesn't violate the caller's expectations.
2. What is the difference between Dependency Inversion (DIP) and Dependency Injection (DI)?
   DIP is the principle (the "what" — decoupling). DI is the technique (the "how" — passing the dependency via constructor or setter). You use DI to achieve DIP.
3. Does the Liskov Substitution Principle (LSP) conflict with the Open/Closed Principle (OCP)?
   No, they do not conflict. In fact, they are complementary. While it may superficially seem like OCP (extending behavior) might lead to an LSP violation (changing behavior), they actually work together to ensure that extensions are safe and predictable.
   The Relationship:
   - OCP tells us how to organize our code so it can grow: by adding new subclasses or implementations instead of modifying existing ones.
   - LSP tells us what those new subclasses are allowed to do: they must adhere to the contract established by the parent class.
   Why the "Conflict" is a Misunderstanding:
   The perceived conflict usually arises from a narrow interpretation of "extension."
   Violation of LSP: If you create a subclass that changes the fundamental "contract" of the parent (e.g., a JsonStorage class that extends XmlStorage but throws an error when XML is expected), you have technically "extended" the system (adhering to OCP), but you have broken the correctness of the system (violating LSP).
   Harmonious Design: To follow both, you must define the parent's contract broadly enough to be extensible. If the parent is DataStorage, then JsonStorage and XmlStorage both fulfill the contract "Store Data" without breaking the expectations of the caller.